

B (2 puncte). Fie următorul log al unei baze de date:

$\langle \text{start_transaction}, T_1 \rangle;$
 $\langle T_1, A, 50, 20 \rangle;$ ✓
 $\langle \text{start_transaction}, T_2 \rangle;$
 $\langle T_2, B, 250, 20 \rangle;$ ✓
 $\langle T_1, A, 40, 50 \rangle;$ ✓
 $\langle T_2, C, 35, 20 \rangle;$ ✓
 $\langle T_2, D, 45, 20 \rangle;$ ✓
 $\langle \text{commit}, T_1 \rangle;$ •
 $\langle \text{start_transaction}, T_3 \rangle;$
 $\langle T_3, E, 55, 20 \rangle;$ ✓
 $\langle T_2, D, 50, 45 \rangle;$ ✓
 $\langle T_2, C, 65, 35 \rangle;$ ✓

$$T_3 \Rightarrow E = 55$$

$$T_2: \begin{aligned} C &= 65 \\ D &= 50 \\ B &= 250 \\ T_1: A &= 40 \end{aligned}$$

$\langle \text{commit}, T_2 \rangle;$ •

$\langle \text{start_transaction}, T_4 \rangle;$

$\langle T_4, F, 100, 20 \rangle;$ ~~$T = 200$~~

$\langle T_4, G, 110, 20 \rangle;$ ~~$B = 110$~~

$\langle \text{commit}, T_3 \rangle;$

$\langle \text{checkpoint} \rangle;$

$\langle T_4, F, 150, 100 \rangle;$ ~~$T = 150$~~

$\langle \text{commit}, T_4 \rangle;$

$\langle T_5, D, 40, 300 \rangle;$

$\langle \text{commit}, T_5 \rangle;$

$\langle T_6, E, 49, 140 \rangle;$

$\langle \text{commit}, T_6 \rangle;$

Presupunând că o intrare în log are forma $\langle \text{XID}, \text{Obiect}, \text{ValoareNouă}, \text{ValoareVeche} \rangle$, care vor fi valorile obiect D, E, F și G salvate pe disc după efectuarea procesului de recuperare a datelor:

- Dacă sistemul este întrerupt chiar înainte de a scrie $\langle \text{start_transaction } T_4 \rangle$ în log?
- Dacă sistemul este întrerupt chiar înainte de a salva în log $\langle \text{commit } T_4 \rangle$?

C (2 puncte). Se dau schemele relaționale: -

- Clienti(CNP, Nume, Prenume) (CNP)

$$\begin{aligned} a) \quad A &= 40 \\ B &= 250 \\ D &= 20 \\ E &= 20 \\ F &= 20 \\ G &= 20 \end{aligned}$$

$$b) \quad \forall T_4 \text{ nu a fost commit} \Rightarrow \text{ROLLBACK } T_4$$

$$\begin{aligned} A &= 40 \\ B &= 250 \\ C &= 65 \\ D &= 50 \\ E &= 55 \\ F &= 20 \\ G &= 20 \end{aligned}$$

C(2puncte). Se dau schemele relaționale: -

- Clienti[CNP, Nume, Prenume]. {CNP} este cheie primară -
- Produse[CodProdus, DenumireProdus]. {CodProdus} este cheie primară și {DenumireProdus} este cheie secundară
- Achizitii[CNP, CodProdus]. {CNP, CodProdus} este cheie primară. {CNP} este cheie externă pentru Clienti, iar {CodProdus} este cheie externă pentru Produse.

Care dintre interogările de mai jos returnează strict numele și prenumele clienților care au cumpărat un produs "DEN" (duplicatele nu se elimină din rezultat)?

a. $\text{SELECT C.NUME, C.PRENUME FROM PRODUSE P LEFT JOIN ACHIZITII A ON P.CODPRODUS=A.CODPRODUS AND P.DENUMIREPRODUS='DEN' RIGHT JOIN CLIENTI C ON A.CNP=C.CNP}$

b. $\text{SELECT C.NUME, C.PRENUME FROM PRODUSE P INNER JOIN ACHIZITII A ON P.CODPRODUS=A.CODPRODUS INNER JOIN CLIENTI C ON A.CNP=C.CNP WHERE P.DENUMIREPRODUS='DEN'}$

c. $\Pi_{\{Nume, Prenume\}} (((\sigma_{DenumireProdus='DEN'} Produse) \otimes_{Produse.CodProdus=Achizitii.CodProdus} Achizitii) \otimes_{Clienti.CNP=Achizitii.CNP} Clienti)$

d. $\Pi_{\{Nume, Prenume\}} (((\sigma_{DenumireProdus='DEN'} Produse) \otimes_{Produse.CodProdus=Achizitii.CodProdus} Achizitii) \triangleleft_{Clienti.CNP=Achizitii.CNP} Clienti)$

e. Niciuna de mai sus. Clienti)

Notă: se consideră că operatorul de proiecție nu elimină duplicatele. Legenda operatorilor utilizați

- Selecție: σ
- Proiecție: Π
- Join condițional: $\otimes$
- Join extern dreapta: $\triangleleft$

D. (2puncte). Aveți tabelele:

- Customers(customer_id, name, country)
- Orders(order_id, customer_id, order_date, total_amount)
- OrderItems(order_item_id, order_id, product_id, quantity, unit_price)
- Products(product_id, product_name, category)

Scrieți o interogare care returnează, pentru fiecare client din Germania, suma totală cheltuită pe produse din categoria "Electronics" în ultimii 2 ani, împreună cu numărul de comenzi distincte efectuate. Coloane din rezultat ar trebui să fie:

customer_id | name | total_spent | order_count

E. (04 puncte) Menționați când este nevoie să se optimizeze un query

1 punct oficiu

```
SELECT
  c.customer_id,
  c.name,
  SUM(oi.quantity * oi.unit_price) AS total_spent,
  COUNT(DISTINCT o.order_id) AS order_count
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN OrderItems oi ON o.order_id = oi.order_id
JOIN Products p ON oi.product_id = p.product_id
WHERE
  c.country = 'Germania' AND
  p.category = 'Electronics' AND
  o.order_date >= CURRENT_DATE - INTERVAL '2 years'
GROUP BY c.customer_id, c.name;
```

E. Optimizăm un query:

- când durează prea mult
- când folosește multe resurse